

EECS 482

Introduction to operating systems

File systems

Peter Chen
University of Michigan

File systems

- File system: a **persistent** data structure
- Persistent across what?
 - Process creation/exit
 - Machine crashes/reboots
 - Power outages
- How to enable persistence across these events?
 - Use persistent storage medium
 - Use persistent pointers (addresses that are stable across reboot), e.g., disk block number
 - Write data in a careful order

Interface to file system

- Create file
- Delete file
- Read <file, offset>
- Write <file, offset>
- Other (e.g., list files in a directory)
- Other persistent data structures
 - SQL database
 - Key-value store

File system workloads

- Optimize data structure for the common case
- Some general rules of thumb
 - Most file accesses are reads
 - Most programs access files sequentially and entirely
 - Most files are small, but most bytes belong to large files

Considerations in storing files

- Need to store metadata (not just file data) persistently
 - File name
 - File size
 - Owner/permissions
 - Time of creation/last access
- For any data structure, need some initial **handle** to access the object
 - We call this handle a **file header** (inode in Unix, Master File Table record in NTFS)
 - File header describes the file and allows you to find its data
- Many data structures are possible

Contiguous allocation

- File is stored in a single contiguous segment on disk
 - File header stores (starting location on disk, size)
 - If file grows, move it to a larger free area
-
- + Fast sequential access: data that is sequential in file space is also sequential in disk space
 - + Easy random access: easy to compute disk location
 - External fragmentation
 - Difficult to grow file

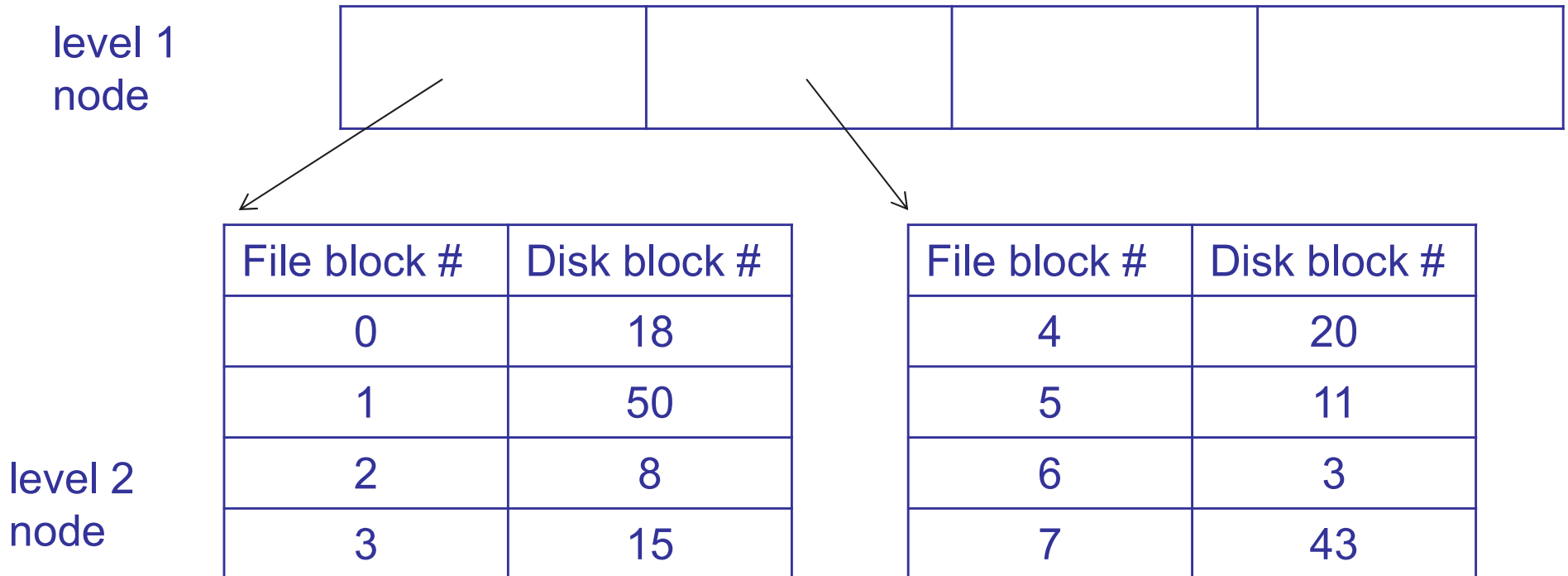
Indexed files

- File header stores an array of block pointers

File block #	Disk block #
0	18
1	50
2	8
3	15

- + Easy to grow file
- + Easy random access
- Potentially slow for sequential access
- How to speed up sequential access?
- How to support large files?

Multi-level indexed files

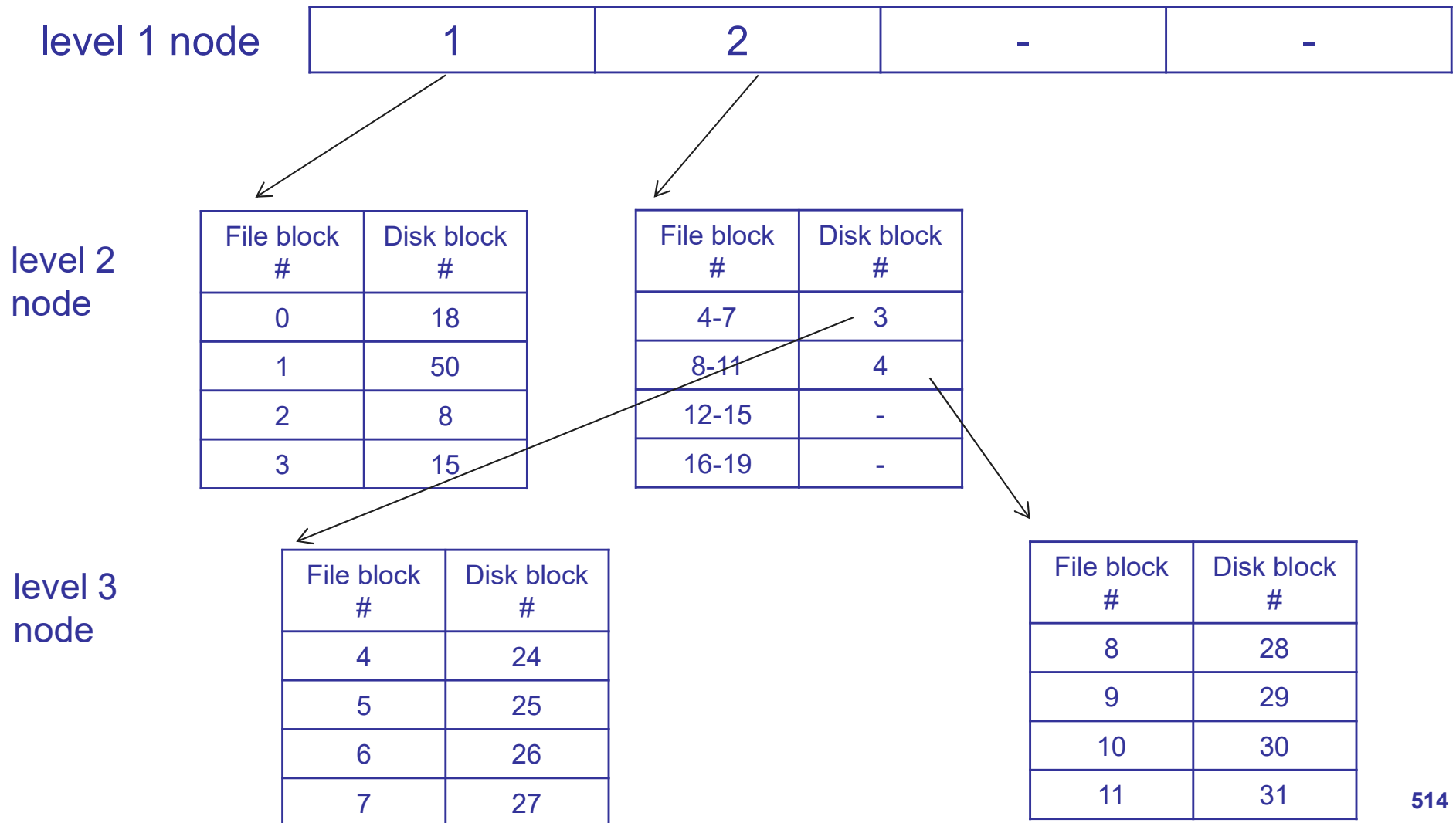


- File header stores (root of) tree of block pointers
- Files can easily grow
- Allows large file, but small files don't waste header space

Multi-level indexed files

- Problems?
- How to fix?

Non-uniform depth



Other data structures

- Can represent file via other dynamic data structures
 - Location of file header should not change as file grows
- Example: linked list
 - What does the file header store?
 - Pros/cons?

Naming and directories

- How do you specify an item in an in-memory data structure?
- How do users specify which file they want to access?
- Once the OS knows the name of the file to access, it must find the header for that file (i.e., it must locate the file header on disk)

Directories

- Directory: data structure to help locate file headers for a set of files
 - Name of file → file header's disk block # for that file
- Must be persistent
- Many data structures possible, e.g., list, hash table, B-tree
- We'll assume a simple table of pairs: (name, file header's disk block #)

Directories and files

- Can often handle directories and files in the same way
 - E.g., use same storage structure (e.g., indexed files)

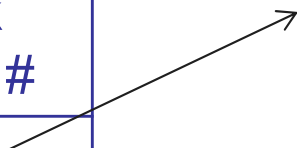
Files

- What data is contained in a file?

inode

File block #	Disk block #
0	18
1	50
2	8
3	15

file data



Hints for Computer System Design
July 1983
Butler W. Lampson
Computer Science Laboratory
Xerox Palo Alto Research Center
Palo Alto, CA 94304

- How do I find a file's data on disk?
 - Contiguous allocation, **indexed files**, multi-level indexed files, linked list, etc.

Directories

- What data is contained in a directory?

inode

Directory block #	Disk block #
0	18
1	50
2	8
3	15

directory data

Name	Disk block # of inode
papers.txt	20
home	5

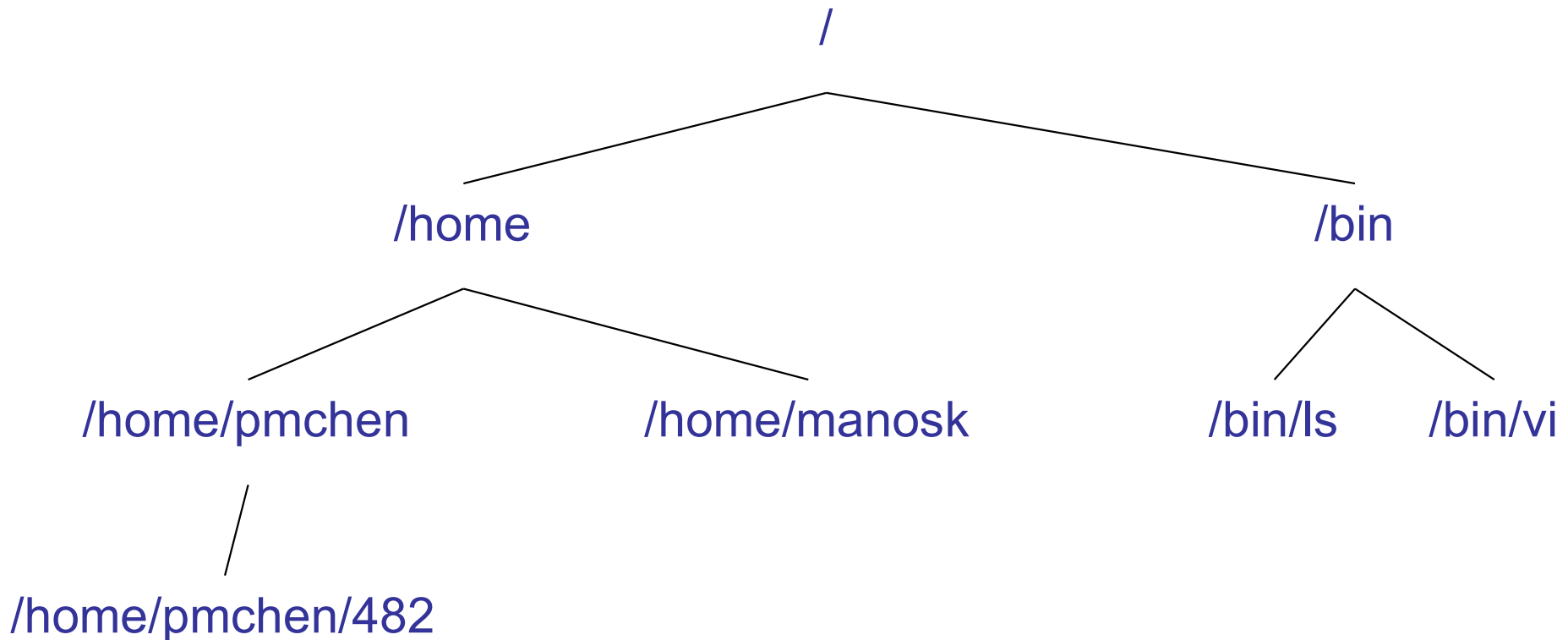
- How do I find a directory's data on disk?
 - Contiguous allocation, **indexed files**, multi-level indexed files, linked list, etc.

Differences between directories and files

- Files: users can read/write arbitrary data
- Directory: can user read/write arbitrary data?
 - Similar to restrictions on entering kernel mode

Structured naming

- How does a file system organize files?



Support for structured naming

- How to map from a hierarchically-structured file name (e.g., /home/pmchen/482/notes) to the disk block # of the file header?

Tree of directories

- What is needed for a tree?
 - Nodes (with one node designated as the root node)
 - Pointers between nodes
 - Exactly one path from root node to each other node
- Hierarchical file system
 - Nodes: directories, files
 - Pointers between nodes: directory entries
- Directory entries
 - File name → disk block # for header
 - Directory name → disk block # for header
 - Another way to treat directories like files

How to read /home/pmchen/482/notes ?

1. Read / inode
2. Read / directory data
3. Read /home inode
4. Read /home directory data
5. Read /home/pmchen inode
6. Read /home/pmchen directory data
7. Read /home/pmchen/482 inode
8. Read /home/pmchen/482 directory data
9. Read /home/pmchen/482/notes inode
10. Read /home/pmchen/482/notes